An Authoritative STM32 Tic-Tac-Toe Server with a Thin Python Client

Themba Shongwe (SHNTHE021) Thembha Hlungwani (HLNTHE010)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—We built a Tic-Tac-Toe game server on the UCT STM32F051C8 board and a Python Tkinter client that talks to it over UART at 115200 baud. The board holds the only copy of the game: it checks every move for game state, range, turn and occupancy before it writes anything, detects all eight winning lines and the draw, and refuses moves once the game is over. The client draws the board only from the server's replies. The newline-terminated ASCII protocol limits frames to 32 bytes, discards incomplete frames after 1000 ms, supports an optional XOR checksum, and answers every bad input with one of nine specific negative acknowledgements. The firmware uses direct register access, and the same game and protocol code also runs in a PC simulator. A pytest harness drives the serial link with no human input. On the board, all 63 tests passed: 48 on the rules, server authority and robustness, and 15 that show the optional AI opponent never makes an illegal move and, on its hardest level, never loses.**

I. Introduction

In a networked game, a client that decides whether its own moves are legal can be modified or simply be buggy, and nothing stops it from corrupting the game. The brief [1] therefore makes the STM32F051 microcontroller the referee: it stores the game, validates every request and returns the authoritative result, while the PC graphical user interface (GUI) only renders confirmed state and forwards player input. Our design guarantees one property above all others: *a rejected request never changes the server's state*.

We chose Tic-Tac-Toe, the brief's baseline game, so that the time went into making the compulsory parts reliable and fully tested rather than into game rules; we then added an artificial-intelligence (AI) opponent, one of the brief's stretch features. We wrote the client in Python with Tkinter [2], because it ships with Python on Windows and needs no installation beyond pySerial [3], and because its event loop combines cleanly with a background reader thread.

Three design decisions shape the system. First, the firmware is split into pure logic (`game.c`, `protocol.c`) and a thin hardware layer (`board.c`) written with direct register access [4], without the hardware abstraction layer (HAL) or CubeMX. Second, every frame is a newline-terminated ASCII line of at most 32 bytes, and every malformed frame gets exactly one specific negative acknowledgement (NACK). Third, because the logic has no hardware dependencies, the same C files also run in a PC simulator, so the automated tests run identically with or without the board.

Task allocation. HLNTHE010: firmware, protocol and board bring-up (all of C). **SHNTHE021**: GUI, tests and AI opponent (all of Python). **Together**: protocol specification, integration on the board, report and video. Our plan, with milestones from the repository's commit history: first GUI (23–25 Oct); firmware, AI opponent, test harness and simulator (26 Oct); integration on the board with all 63 tests passing, then the README and diagrams (26 Oct); report and video (by the 27 Oct deadline).

Structure. Section II specifies the message protocol and its departures from the brief. Section III presents the architecture, the server finite-state machine (FSM), the GUI flowchart and the circuit. Section IV explains selected code excerpts. Section V argues that the server is authoritative and that rejected requests change nothing. Section VI covers malformed input, and Section VII reports the test results. Section VIII concludes, and Sections IX and X link the video and the code.

TABLE I
FRAMING RULES AND THE RESPONSE TO EACH VIOLATION.

Rule	Server behaviour
Terminator	`\n` or `\r`; an empty frame gets no reply.
Maximum length	32 bytes; a longer frame is dropped up to its terminator, then `NACK:TOOLONG`.
Incomplete frame	No byte for 1000 ms: dropped, `NACK:TIMEOUT`.
Invalid character	Byte outside 0x20–0x7E: `NACK:BADCMD`.
UART error	Overrun, framing or noise error, or full buffer: `NACK:BADCMD`.
Checksum (optional)	Suffix `*HH`, XOR of all bytes before `*` in upper-case hex (e.g. `NEW*5C`); mismatch gives `NACK:CHECKSUM`.

II. Requirements and Message Protocol

A. Newline-terminated lines of at most 32 bytes frame the stream

The link is USART1 of the STM32F051 (PA9 TX, PA10 RX) at 115200 baud, 8 data bits, no parity, 1 stop bit and no flow control. It reaches the PC through the board's ST-Link virtual COM port. A frame is a line of printable ASCII (0x20–0x7E) ending in `\n` or `\r`, so `\r\n` also works (the empty frame between the two is ignored, like any blank line) and so does the Enter key of a terminal program such as PuTTY, which sends only `\r`. Every non-blank frame gets exactly one reply line. Table I lists each framing rule and the server's response when a frame breaks it.

B. Nine characters encode the board row by row

The board is nine characters for cells 0–8, row by row, with `.` for an empty cell and `X` or `O` for a mark. X always moves first. For example, after X plays the centre and O the top-left corner, the server replies `OK:BOARD:O...X....`: cell 0 holds O and cell 4 holds X.

C. Nine commands cover a whole session

Table II lists every request, the states that accept it and the reply on success. A winning move returns `WIN:X` or `WIN:O` and the ninth move without a line returns `DRAW`; both end the game. The server can also change state without a request: pressing the board's button abandons any game and the server sends the unsolicited line `EVT:RESET`, so the client learns about it and clears its view.

TABLE II
COMMAND SET. † MARKS A DEPARTURE FROM THE BRIEF.

Request	Accepted in	Success response
`CONNECT`	any state†	`OK`; no state change
`NEW`	IDLE, GAME_OVER	`OK:BOARD:.........`; X to move, IN_PROGRESS
`MOVE:<i>`	IN_PROGRESS	`OK:BOARD:<9>`, or `WIN:X`, `WIN:O`, `DRAW` (GAME_OVER)
`STATE`	IN_PROGRESS, GAME_OVER	`BOARD:<9>`
`RESIGN`	IN_PROGRESS	`OK:GAMEOVER`; GAME_OVER
`DISCONNECT`	IDLE†, GAME_OVER	`OK`; board cleared, IDLE
`MOVE:<P>:<i>`†	IN_PROGRESS	as `MOVE:<i>`, but P (X or O) must be the player to move
`EVT:RESET`†	sent by server	button pressed; game abandoned, IDLE

D. Each error names one cause and changes nothing

Table III lists every error. Each one leaves the board, the turn and the state exactly as they were (Section V argues why). A `MOVE` that fails several checks gets the first failure in this order: game in progress (`NOGAME`), argument well formed (`BADCMD`), index in range (`RANGE`), right player (`NOTURN`), cell empty (`OCCUPIED`). The order makes every request's reply unique, so the tests can assert it exactly.

TABLE III
EVERY ERROR RESPONSE AND WHEN THE SERVER SENDS IT.

Response	Sent when
`NACK:BADCMD`	unknown command, wrong case, malformed `MOVE` argument (e.g. `MOVE:a`, `MOVE: 1`), non-printable byte or UART error
`NACK:NOGAME`	`MOVE` or `RESIGN` with no game in progress; `STATE` in IDLE
`NACK:RANGE`	`MOVE` index outside 0–8 (e.g. 9, 42, −1)
`NACK:OCCUPIED`	`MOVE` to a cell that is not empty
`NACK:NOTURN`	`MOVE:<P>:<i>` where P is not the player to move
`NACK:STATE`†	`NEW` or `DISCONNECT` during a game
`NACK:TOOLONG`†	frame longer than 32 bytes
`NACK:TIMEOUT`†	incomplete frame idle for 1000 ms
`NACK:CHECKSUM`†	checksum suffix present but wrong

E. Seven departures make every rule testable

1) **`MOVE:<P>:<i>` names the player.** With one client sending plain `MOVE:<i>`, the server always plays the side to move, so no move can be out of turn. Naming the player makes `NACK:NOTURN` reachable and testable; the GUI uses this form for AI moves. Plain `MOVE:<i>` still works.
2) **`CONNECT` works in every state**, so a client that restarts mid-game can reconnect and resynchronise with `STATE`.
3) **`DISCONNECT` also works in IDLE**; during a game it is refused with `NACK:STATE`, so a game cannot vanish by accident.
4) **Four extra errors** (`STATE`, `TOOLONG`, `TIMEOUT`, `CHECKSUM`) make every failure specific.
5) **`EVT:RESET`** reports the button reset to the client.
6) **`WIN` and `DRAW` carry no board**, as in the brief; the client sends `STATE` to draw the final board.
7) **`\r` also ends a frame**, so a person can type commands in any terminal; the brief only asks for a defined terminator.

Fig. 1 shows a typical session. The second `MOVE:4` is rejected and the next `STATE` would return the same board as before it.

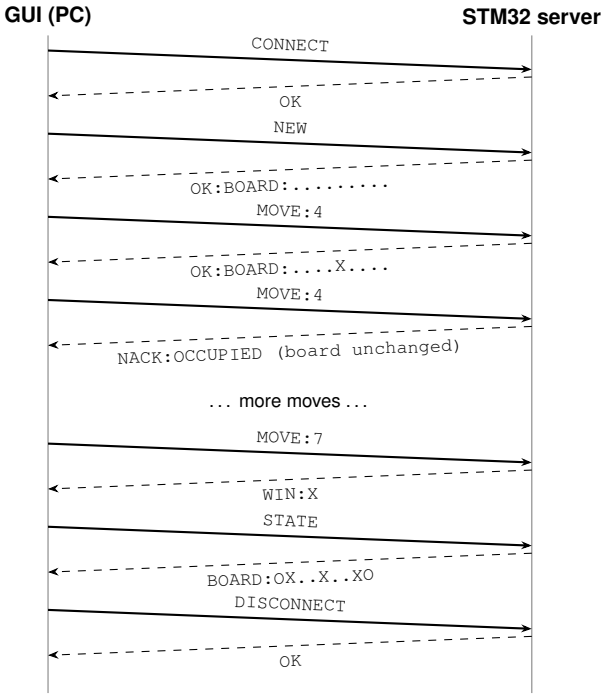


Fig. 1. A session from connection to disconnection. The occupied-cell move is rejected and leaves the board unchanged; after `WIN:X` the client fetches the final board with `STATE`. Requests are solid, replies dashed; time runs downwards.

III. SPECIFICATION AND DESIGN

A. Every rule lives in hardware-free C on the server

Table IV shows how the work is divided. Only `board.c` touches registers; `game.c` and `protocol.c` take bytes and

a millisecond time in and give reply strings out. This split has two benefits: the rules can be read and argued about without hardware detail, and the identical files compile into the PC simulator (`sim/`), which serves them on a pseudo-terminal.

The server owns every decision that affects the outcome: the board, whose turn it is, whether a move is legal, and whether the game is won or drawn. The client owns only presentation (drawing, names, scores, themes) and the AI's choice of move, which it must still submit to the server like any human move. A client that is wrong or hostile can therefore cause, at worst, a rejected request.

TABLE IV
MODULES AND WHAT EACH ONE OWNS.

Module	Responsibility
`game.c`	Board, turn, state and result; move checks; win and draw detection. Pure C.
`protocol.c`	Frame assembly, timeout, checksum, command parsing and replies. Pure C.
`board.c`	Registers: 48 MHz clock, GPIO, USART1 with a receive interrupt and ring buffer, SysTick, LCD.
`main.c`	Non-blocking super-loop: bytes to the protocol, timeouts, button debounce, LED pattern.
`client.py`	Tkinter GUI; a reader thread and a reply queue; draws only confirmed boards.
`ai.py`	Suggests a move (Easy, Medium, Hard); never changes the game itself.

B. Three states, and a five-check branch guards every move

Fig. 2 shows the server FSM. At power-on the server is in IDLE. `NEW` starts a game in IN_PROGRESS, where each `MOVE` passes through the validation branch on the right. A move that fails any check exits with its NACK and returns to IN_PROGRESS with nothing changed. A move that passes all five is written to the board; the server then checks whether the mover completed a line (`WIN`), whether all nine cells are full (`DRAW`), and otherwise switches the turn. `RESIGN`, a win and a draw all lead to GAME_OVER, which accepts `NEW`, `STATE` and `DISCONNECT` and refuses moves with `NACK:NOGAME`. The button returns the server to IDLE from either game state. For clarity, the figure omits `CONNECT` (accepted everywhere, no change), `STATE` (no change) and the out-of-state NACKs of Table III.

C. The GUI waits for replies without blocking

Fig. 3 shows the client. The Tk event loop runs on the main thread. After *Connect* opens the port, a background thread reads lines and puts them on a queue; a timer drains the queue every 20 ms. Each sent command is appended to a list of pending requests, and each reply is matched to the oldest one. A reply that matches the board pattern is the only thing that redraws the board; a NACK shows its reason in plain words, and `WIN` or `DRAW` locks the cells, updates the scoreboard and fetches the final board with `STATE`. A line starting with `EVT:` is an unsolicited event and is never matched to a request. If no reply arrives within 2 s the GUI says so; if the cable is unplugged the reader thread reports the error and the GUI closes the link cleanly. In AI mode, after each confirmed board the GUI asks `ai.py` for a move and sends it 500 ms later, so the player can follow the game.

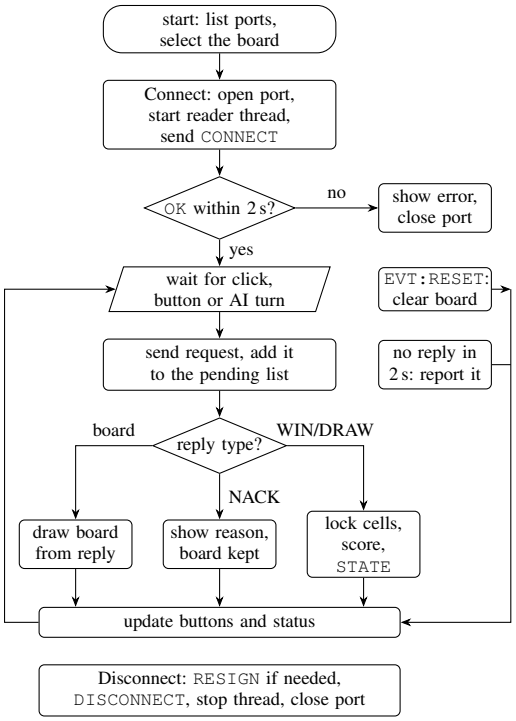


Fig. 3. GUI flowchart. The board is redrawn only from a server reply that contains a board; NACKs, timeouts and events never redraw it.

D. The ST-Link's virtual COM port carries the UART

Fig. 4 shows the hardware. USART1 TX (PA9) and RX (PA10) connect to the board's ST-Link, which appears on the PC as a USB virtual COM port, so no separate USB-UART converter is needed and the ground is shared through the USB cable. Button SW0 on PA0 pulls the pin to ground when pressed; the firmware enables the internal pull-up, debounces it for 30 ms and treats a press as a local reset to IDLE. LED D0 on PB0 (active high, 150 Ω series resistor) shows the game: off in IDLE, solid when X is to move, blinking at 1 Hz when O is to move and at 5 Hz after the game ends. The firmware also clears the 16-character LCD at start-up, so text left by an earlier program does not confuse the user.

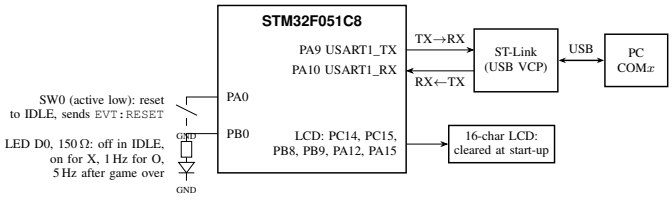


Fig. 4. Hardware connections: USART1 reaches the PC through the ST-Link's virtual COM port; SW0 resets the game; LED D0 shows whose turn it is and when the game is over.

IV. IMPLEMENTATION

The listings below are trimmed: comments and some declarations are removed, and nothing else is changed.

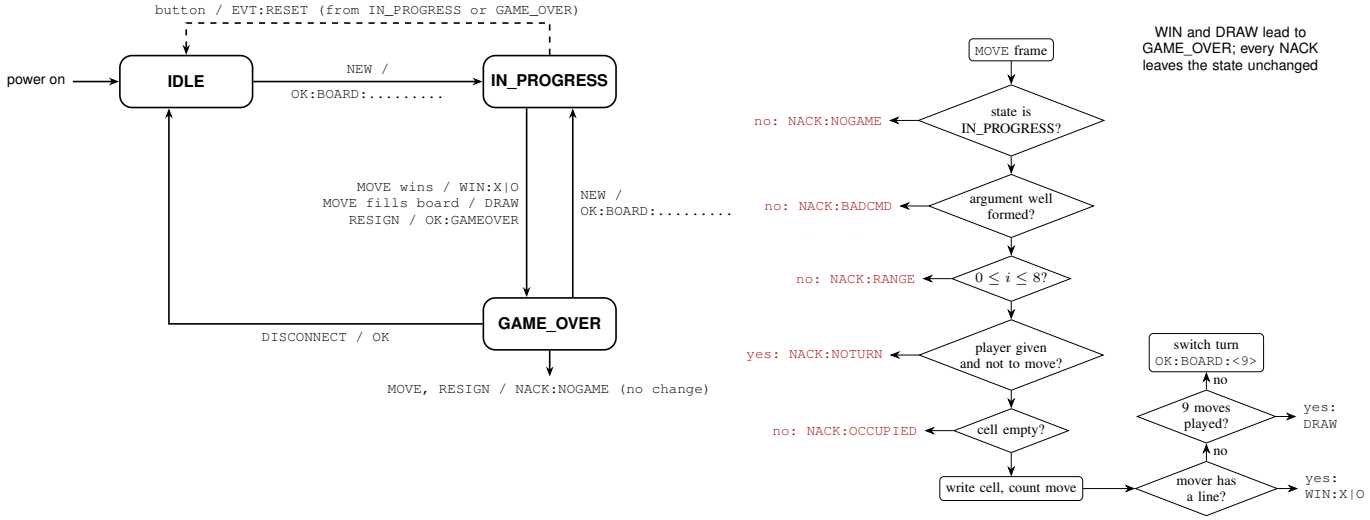


Fig. 2. Server FSM (left) and the move-validation branch (right). Every check runs before the only write, so a move that ends in a NACK leaves the board, the turn and the state unchanged. Transitions are labelled "request / reply".

A. *The receive interrupt only queues bytes, and flags any loss*

At 115200 baud a byte arrives every 87 µs. If the main loop polled the USART, a byte arriving while it was busy (for example sending a reply) would overrun the single receive register and be lost silently. Listing 1 instead moves each byte into a 64-byte ring buffer inside the interrupt and does nothing else; all parsing happens in the main loop. The less obvious part is the error path. On the STM32F0 an overrun, framing or noise error keeps the interrupt pending until its flag is cleared in `USART_ICR` [4], so the handler clears the flags (otherwise it would fire forever) and sets `rx_error`. The main loop then feeds a 0x00 byte into the framer, which marks the current frame bad, so a frame with a missing byte is answered with `NACK:BADCMD` instead of being executed as a different, wrong command. A full ring buffer is treated the same way.

```
static void uart_irq(void)
{
    uint32_t isr = USART_ISR(UART_BASE);

    if (isr & USART_ICR_ERRORS) {
        USART_ICR(UART_BASE) = USART_ICR_ERRORS;
        rx_error = 1;                   /* frame damaged */
    }
    if (isr & USART_ISR_RXNE) {
        uint8_t  b    = (uint8_t)USART_RDR(UART_BASE);
        uint16_t next = (uint16_t)((rx_head + 1u)
                                   & (RX_RING_SIZE - 1u));
        if (next == rx_tail) {
            rx_error = 1;               /* ring full */
        } else {
            rx_ring[rx_head] = b;
            rx_head = next;
        }
    }
}
```

Listing 1. UART receive interrupt (`board.c`).

B. *The framer drops an oversized frame whole*

Listing 2 assembles frames one byte at a time. The obvious approach, cutting a long line at 32 bytes, is dangerous: the remainder of the line would then be parsed as a new frame, and a long line could contain a valid command. Instead, once a frame passes `FRAME_MAX` the framer sets `overflow` and skips every byte up to the next `\n`, then replies `NACK:TOOLONG` once. Non-printable bytes are replaced by `?` but still counted, so a long binary burst still trips the length limit. Every byte also records its arrival time in `last_ms`; a separate `server_poll()` call in the main loop discards a frame that has been idle for 1000 ms. Because the time comes in as an argument, the same code runs on SysTick on the board and on a monotonic clock in the simulator.

```
int server_feed(server_t *s, uint8_t byte, uint32_t now_ms,
                char *resp, size_t resp_len)
{
    framer_t *f = &s->framer;
    f->last_ms = now_ms;
    if (byte == '\n' || byte == '\r') {
        int handled = 1;
        if (f->overflow) {
            reply(resp, resp_len, "NACK:TOOLONG");
        } else if (f->bad_char) {
            reply(resp, resp_len, "NACK:BADCMD");
        } else if (f->len == 0) {
            handled = 0;                /* blank line */
        } else {
            f->buf[f->len] = '\0';
            server_handle_frame(s, f->buf, resp, resp_len);
        }
        framer_clear(f);
        return handled;
    }
    if (f->overflow)
        return 0;                       /* skip to '\n' */
    if (f->len >= FRAME_MAX) {
        f->overflow = 1;
        return 0;
    }
    if (byte < 0x20 || byte > 0x7E) {
        f->bad_char = 1;                /* keep counting */
        byte = '?';
    }
    f->buf[f->len++] = (char)byte;
    return 0;
}
```

Listing 2. Frame assembly (`protocol.c`).

C. A reader thread keeps the GUI responsive

Tkinter widgets may only be touched from the thread that runs the event loop, and a blocking `read()` on that thread would freeze the window while the board is silent. Listing 3 moves reading to a daemon thread that splits the byte stream into lines and puts them on a thread-safe queue; the Tk thread drains the queue every 20 ms with `after()`. The handler then separates the two kinds of line: `EVT:` lines are unsolicited and never consume a pending request, and a reply that arrives when no request is pending is logged but never drawn. Matching replies to requests in order (oldest first) is valid because the server answers every frame exactly once, in order.

```
def run(self):                    # reader thread
    buf = b""
    while self.running:
        try:
            chunk = self.ser.read(64)
        except (serial.SerialException, OSError,
                TypeError) as exc:
            if self.running:
                self.out.put(("error", str(exc)))
            return
        buf += chunk
        while b"\n" in buf:
            line, buf = buf.split(b"\n", 1)
            text = line.decode("ascii", errors="replace")
            self.out.put(("line", text.strip("\r")))

def _on_line(self, line):         # Tk thread
    self._log(f"< {line}")
    if line.startswith("EVT:"):
        self._on_event(line)
        return
    if not self.pending:
        return                    # stray: never redraw
    cmd = self.pending.popleft()
    self._on_reply(cmd, line)
```

Listing 3. Reader thread and reply dispatch (`client.py`).

D. The test harness never changes the port's timeout

In our first run on the board, 30 of the 47 tests failed and 15 more errored, with replies missing characters, e.g. `NACKNOGAME`. We first suspected the baud rate, because at 8 MHz the nearest divider gives 115942 baud (+0.64 %); we moved the clock to the 48 MHz phase-locked loop (PLL), which gives 115108 baud (−0.08 %). The failures continued. The real cause was in the harness: it changed pySerial's `timeout` before each read, and on Windows that reconfigures the port, which made the ST-Link's virtual COM port drop bytes already in flight. Listing 4 shows the fix: the timeout is set once when the port is opened (50 ms), and `read_line()` polls against its own deadline instead. With this change all 47 tests in the suite at that time passed on the board. We kept the 48 MHz clock for its larger margin against the internal oscillator's tolerance.

```
def _pull(self):
    # waits at most POLL_S (the timeout set at open)
    self.buf += self.ser.read(self.ser.in_waiting or 1)

def read_line(self, timeout=READ_TIMEOUT_S) -> str:
    deadline = time.monotonic() + timeout
    while b"\n" not in self.buf:
        assert time.monotonic() < deadline, \
            f"no complete reply within {timeout}s"
        self._pull()
    line, self.buf = self.buf.split(b"\n", 1)
    return line.decode("ascii").rstrip("\r")
```

Listing 4. Polling reads with a fixed port timeout (`conftest.py`).

V. Game Logic and Server Authority

A. A move passes five checks before the first write

A `MOVE` is checked in two places. `handle_move()` in `protocol.c` rejects a move with no game in progress (`NACK:NOGAME`) and then parses the argument into a local player and index (`NACK:BADCMD` if it is not `<n>` or `<P>:<n>` with at most three digits). `game_move()` in Listing 5 then checks the state again, the range, the turn and the cell. Its first half only reads; its second half, the commit, starts at the first assignment, and is reached only when every check has passed.

```
move_status_t game_move(game_t *g, int index, char player)
{
    /* 1. Checks only. */
    if (g->state != ST_IN_PROGRESS)
        return MV_NOGAME;
    if (index < 0 || index >= BOARD_CELLS)
        return MV_RANGE;
    if (player != 0 && player != g->turn)
        return MV_NOTURN;
    if (g->cells[index] != CELL_EMPTY)
        return MV_OCCUPIED;
    /* 2. Commit. */
    char mover = g->turn;
    g->cells[index] = mover;
    g->moves++;
    if (game_winner(g->cells) == mover) {
        g->result = (mover == 'X') ? RES_WIN_X : RES_WIN_O;
        g->state  = ST_GAME_OVER;
        return MV_WIN;
    }
    if (g->moves == BOARD_CELLS) {
        g->result = RES_DRAW;
        g->state  = ST_GAME_OVER;
        return MV_DRAW;
    }
    g->turn = (mover == 'X') ? 'O' : 'X';
    return MV_OK;
}
```

Listing 5. Check, then commit (`game.c`).

B. No path that ends in an error writes the game state

The game state is the `game_t` structure: nine cells, the turn, the state, the result and the move count. The argument has four steps, each of which can be checked against the code.

1) **Only five functions in `game.c` write `game_t`**: `game_init` (at power-on and inside `game_reset`), `game_new`, `game_move` (its commit half), `game_resign` and `game_reset`. No code in `protocol.c` or `main.c` assigns to its fields, and the client can reach the server only through frames.
2) **Framing errors never reach a command.** In `server_feed()` (Listing 2), `TOOLONG`, `BADCMD` for bad bytes, and `TIMEOUT` (in `server_poll()`) reply and clear the framer without calling `server_handle_frame()`, so no writer can run.
3) **In `server_handle_frame()` each error returns before its writer.** A wrong checksum returns first,

and `strip_checksum()` edits only a local copy of the frame. `NEW` calls `game_new` only in the branch where the state is not IN_PROGRESS; the other branch sends `NACK:STATE`. `RESIGN` and `DISCONNECT` are guarded the same way. `STATE` and `CONNECT` call no writer at all, and an unknown command falls through to `NACK:BADCMD`.

4) **A rejected move returns before the commit.** In `handle_move()` the `NOGAME` and `BADCMD` paths return before `game_move()` is called, and `parse_move()` writes only local variables. In `game_move()` every `NACK` status is returned by one of the four checks, all of which precede the first assignment.

Hence every reply that starts with `NACK:` leaves `game_t` exactly as it was. The only state change without a request is the button reset, which is a deliberate local action. The test `test_no_rejected_request_changes_state` checks this on the running server: mid-game it sends eleven bad requests (occupied, out of range, out of turn, empty, non-numeric and four-digit moves, `NEW`, `DISCONNECT`, wrong case, an unknown command and a bad checksum), checks after each one that `STATE` returns the same board, and finally checks that the next legal move is still accepted for the correct player.

C. The server alone decides whose turn it is

The server keeps the turn in `g->turn`. A plain `MOVE:<i>` is always played for that player, so a client cannot play for the wrong side, and the turn changes only in the last line of the commit. A `MOVE:<P>:<i>` names the player; if P is not the player to move, the server replies `NACK:NOTURN` and changes nothing (`test_out_of_turn_move_is_rejected`). The GUI sends AI moves in this form, so even a faulty AI cannot move for the human.

D. Eight lines are checked after every move

`game_winner()` checks a table of the eight winning lines (three rows, three columns, two diagonals). It is called after the move is written, and the win check comes before the draw check. This order matters: a ninth move that completes a line is a win, not a draw. Only the mover can have completed a line, because the opponent's position did not change. On a win or a draw the state becomes GAME_OVER, so every later `MOVE` fails the first check with `NACK:NOGAME` until `NEW` (`test_move_after_game_over_is_rejected`, `test_every_win_line_is_detected`).

E. The client holds no rules

The client never decides whether a move is legal. It sends the cell that was clicked and redraws only from a reply that contains a board; it does not predict the result of a move. It does look at a confirmed board, but only to present it: to highlight the winning line after the server has sent `WIN`, and to ignore clicks while the AI is to move. If the client contained rules, a disagreement between its copy and the server's would show a board the server never accepted, and a modified client could skip them. The AI opponent follows the same rule: `ai.py` only suggests a cell, and the server validates it like any human move.

VI. Protocol Robustness and Integrity

The server answers every kind of bad input with exactly one specific NACK, discards it completely, and parses the next frame as if nothing had happened. Table V maps each kind of bad input to its handling and to the test that checks it; each of those tests also sends a valid command afterwards and checks that the link has recovered.

TABLE V
BAD INPUT, THE SERVER'S HANDLING, AND THE TEST THAT CHECKS IT.

Bad input	Handling and response	Test (`test_...`)
Partial frame	discarded after 1000 ms idle; `NACK:TIMEOUT`	`partial_frame_times_out`
Oversized frame	skipped to its `\n`; `NACK:TOOLONG` once	`oversized_frame_is_rejected_then_link_recovers`
Garbage bytes	bad bytes mark the frame; `NACK:BADCMD` at `\n`	`garbage_bytes_are_rejected_then_link_recovers`
Wrong case or unknown command	exact string match fails; `NACK:BADCMD`	`malformed_commands_get_badcmd` (9 cases)
Corrupted bytes (checksum)	XOR mismatch or bad suffix; `NACK:CHECKSUM`	`bad_checksum_is_rejected`, `valid_checksum_is_accepted`
UART errors (overrun, framing)	ISR clears flags; frame poisoned; `NACK:BADCMD`	none: cannot be injected from the PC
Split frame, CR, CRLF, blank lines	assembled across writes; `\r` or `\n` ends a frame; empty frames get no reply	`frame_split_across_writes_is_assembled`, `cr_alone_ends_a_frame`, `crlf_is_accepted`, `blank_lines_are_ignored`

A. An idle partial frame is cleared after one second

A frame with no new byte for 1000 ms is discarded and answered with `NACK:TIMEOUT`. The value is long compared with a byte time at 115200 baud (87 µs) and with the pauses of a person typing a command by hand in a terminal, yet short enough that a client that lost part of a frame recovers within a second. The test sends `MOV` with no terminator, expects `NACK:TIMEOUT` within 3 s, and then checks that `NEW` is parsed on its own, which shows that the stale bytes were really discarded.

B. Garbage and oversized input resynchronise on the next newline

The newline is the only synchronisation point. A frame containing any byte outside 0x20–0x7E is marked bad, so the test's burst of control and high-bit bytes (`0x00 0x01 ESC [2J 0x7F 0xFF 0xFE 0x80`) is answered with a single `NACK:BADCMD`, and the next `NEW` succeeds. An oversized frame (a 205-byte `MOVE`) gives exactly one `NACK:TOOLONG`; the test then checks that the server stays silent, so the tail of the frame was not parsed as further commands, and that

CONNECT still works. Commands match only exactly, so move:1, MOVE: 1 and CONNECT (with a trailing space) are all rejected rather than guessed at.

C. Every loop in the firmware is bounded

The main loop never waits for input: it takes whatever bytes are in the ring buffer (at most 64 per pass), checks the frame timeout, debounces the button by comparing timestamps, and updates the LED. Transmitting a reply waits for the transmit register, but each byte's wait is capped at 100 000 polls, so a stuck UART drops the reply instead of hanging the server. The longest reply (OK:BOARD: and nine cells, 19 bytes) takes about 1.7 ms to send, while the ring buffer holds 64 bytes (5.6 ms of continuous input), so input is not lost while a reply is sent. The waits during clock start-up are also bounded; if the PLL fails to lock, the firmware stays on the 8 MHz internal oscillator and derives the baud rate from that clock. The only fixed delay is the roughly 60 ms LCD initialisation at power-on, before the server starts. We did not use the watchdog: no code path waits without a bound, so there is nothing for it to catch.

VII. Validation

A. pytest drives the real serial link with no human input

The harness uses pytest [5] and pySerial [3]. It opens the port given by --port (or --port auto, which finds the board's ST-Link by its USB vendor ID), and every test starts from a known state: a fixture discards any pending input, ends any game with RESIGN and DISCONNECT, and checks that CONNECT returns OK, which puts the server in IDLE. Without a port, the harness builds the PC simulator from the same game.c and protocol.c and runs the identical tests against it. The command is python -m pytest tests --port auto. The 63 tests are in three groups: the brief's required cases and server authority (test_server.py, 48 tests including all eight win lines), robustness (part of the same file), and the AI opponent (test_ai.py, 15 tests).

B. All 63 tests pass on the board

Table VI maps each required case from the brief to its test. On the board (UCT STM32F051C8, 48 MHz, ST-Link virtual COM port, Windows, Python 3.14), with the final firmware, the full suite gave:

```
63 passed in 32.14s
```

The 8 offline AI tests run on the PC alone; the other 55 exchange frames with the board. The same 63 tests also pass on the simulator. Together these results cover more than the required cases: every win line, out-of-turn and out-of-state requests, the no-change argument of Section V, and every row of Table V except UART hardware errors. For the AI, the offline tests explore every possible opponent reply and show that Hard never loses as X or as O, and the on-server tests play 30 games (three levels, both sides, five games each) in which the server accepts every AI move.

TABLE VI
THE BRIEF'S REQUIRED TESTS AND THEIR RESULTS.

Requirement (brief, p. 4)	Test (test_...)	Result
CONNECT returns OK; NEW returns an empty board	connect_ok_and_new_ returns_empty_board	pass (board)
A legal move changes the board exactly as expected	legal_move_changes_ board_exactly	pass (board)
Occupied cell returns NACK:OCCUPIED, board unchanged	occupied_cell_is_ rejected_and_board_ unchanged	pass (board)
Out-of-range move returns NACK:RANGE	out_of_range_move_is_ rejected (9, 10, 99, −1)	pass (board)
Move after game over returns NACK:NOGAME	move_after_game_over_is_ rejected	pass (board)
Scripted win and draw give the correct result	scripted_win_for_x, _for_ o, scripted_draw	pass (board)
AI never makes an illegal move	ai_never_makes_an_ illegal_move (6 cases)	pass (board)

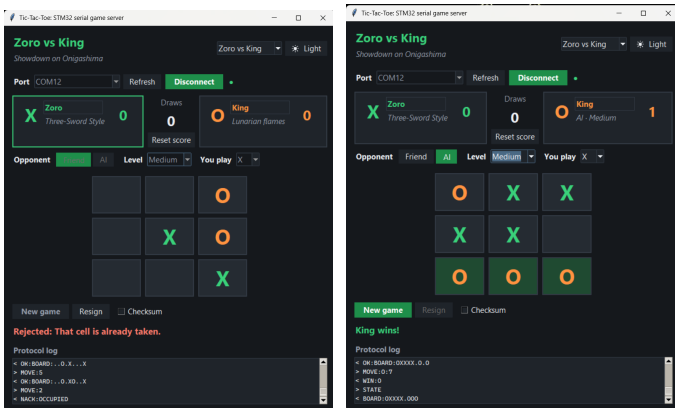


Fig. 5. The final GUI connected to the board. Left: mid-game, X clicks the occupied cell 2; the board replies NACK:OCCUPIED, the user sees "Rejected: That cell is already taken.", the board is unchanged and it is still X's turn. Right: against the Medium AI, which plays O; its moves are sent as MOVE:O:<i>, and the winning row is highlighted and the score updated only after the board's WIN:O.

Fig. 5 shows the system working on the board. In the left screenshot the client does not check the move itself: it sends MOVE:2, and the rejection, its wording and the unchanged board all follow from the server's NACK:OCCUPIED. In the right screenshot the AI's move is validated by the board like any human move, and after WIN:O the client asks for the final board with STATE. Both also show the player names, the scoreboard and the always-visible protocol log.

C. Testing on the board exposed faults the simulator could not

These appeared only on the real board. The first was the dropped-byte fault of Listing 4: the simulator's pseudo-terminal does not reconfigure itself when the timeout changes, so all tests passed there while 45 failed or errored on the board. Fixing the harness, not the firmware, brought the board to 47 of 47, the size of the suite at that time; the remaining tests were added afterwards. The second was in the build: moving the source folders into Core/ silently removed them from the CubeIDE project's source list, so a clean build would have compiled only the startup file. We found it by checking the project file, and fixed the source and

include paths. Typing commands by hand in PuTTY exposed a usability gap: PuTTY's Enter key sends only `\r`, which the server then ignored, so every typed command ended in `NACK:TIMEOUT`. We made `\r` a terminator as well and added `test_cr_alone_ends_a_frame`. Writing the tests also exposed a fault in a test: our first garbage test was longer than 32 bytes, so the server correctly answered `NACK:TOOLONG` rather than `NACK:BADCMD`; we shortened the input so that the test checks what its name says.

VIII. Conclusion

We built an STM32F051 Tic-Tac-Toe server that is the only authority on the game, and a Tkinter client that only displays what the server confirms. Every compulsory requirement is met and backed by a named test: the three-state FSM, specific errors for out-of-range, occupied, out-of-turn and out-of-state moves, detection of all eight win lines and the draw, and safe handling of partial, oversized and garbage input. All 63 tests, including the AI tests, passed on the board on 26 October. The code is structured so that every check precedes the first write, which is why no rejected request can change the state.

Usefulness. The same pattern applies wherever a microcontroller must stay in control of a device while a PC or phone only requests actions, such as a lab instrument, a door controller or a vending machine: line framing with a length limit and a timeout, specific error replies, and check-then-commit handlers. Keeping the logic free of hardware calls let us test it without a board, and the same harness then confirmed it on the board.

Limitations. The server serves one client over one link; it has no notion of separate players' identities, so whoever holds the port can play both sides. The checksum is optional and an 8-bit XOR misses some errors, such as two flipped bits in the same position. Replies carry no checksum. Recovery from UART hardware errors is implemented but not covered by an automated test, because the PC cannot inject them. The scoreboard and player names live only in the GUI and are lost when it closes.

Improvements. Make the checksum mandatory and use a CRC-8 on both requests and replies; add two-client play with a session token per player; run the AI on the STM32 so the board can be a standalone opponent; use the LCD to show the board and the result; and add a hardware-timer move clock, another of the brief's stretch features.

IX. Video

The demonstration video is available at https://uctcloud-my.sharepoint.com/:v:/g/personal/hlnthe010_myuct_ac_za/IQBYzcD4Rq2CQZ0Q19iTq5doAW8ZQiqHM4TkzdgaYsX_Xi4?e=OtsxWH. In order, it shows connecting to the board, legal moves, a rejected move, a win or a draw, moves disabled after the game is over, a new game, the button and LED behaviour, a game against the Hard AI, and disconnecting.

X. Code

The source code is at https://github.com/Mtimande23/EEE3095-PROJECT. The repository is the STM32CubeIDE project itself (firmware in `Core/`), together with the GUI (`gui/`), the test harness (`tests/`), the optional PC simulator (`sim/`) and the protocol specification (`docs/`); the `README.md` gives the setup, build, flash, run and test instructions.

References

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS project 2026: Serial game server," Course project brief, 2026.

[2] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," https://docs.python.org/3/library/tkinter.html, python 3.14.

[3] C. Liechti, "pySerial documentation," https://pyserial.readthedocs.io/.

[4] *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced Arm-based 32-bit MCUs, Reference Manual*, STMicroelectronics.

[5] pytest-dev team, "pytest documentation," https://docs.pytest.org/.